

PRACTICAL : WEEK 9

Task -1 :

Implement a file copy utility using low-level Linux I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

CODE:

filter_copy.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 4096

void low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    src = open(source, O_RDONLY);

    if (src < 0)
    {
        perror("Error opening source file");
        exit(1);
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0)
    {
        perror("Error opening destination file");
        close(src);
        exit(1);
    }

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("Write error");
            close(src);
            close(dest);
            exit(1);
        }
    }

    if (bytes_read < 0)
        perror("Read error");
}
```

```

    if (bytes_read < 0)
        perror("Read error");

    /* Demonstrate lseek() */
    lseek(src, 0, SEEK_SET);

    close(src);
    close(dest);
}

void library_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    src = fopen(source, "rb");

    if (src == NULL)
    {
        perror("Error opening source file");
        exit(1);
    }

    dest = fopen(destination, "wb");

    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        exit(1);
    }

    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        fwrite(buffer, 1, bytes_read, dest);
    }

    fclose(src);
    fclose(dest);
}

```

```

GNU nano 8.7.1

    fclose(src);
    fclose(dest);
}

int main()
{
    const char *source = "source.txt";

    clock_t start, end;
    double low_time, library_time;

    printf("=== File Copy Performance Test ===\n\n");
    printf("Low-level I/O copy started...\n");
    start = clock();
    low_level_copy(source, "low_level_copy.txt");
    end = clock();

    low_time = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Low-level copy completed.\n");
    printf("Time taken: %.6f seconds\n\n", low_time);

    printf("Standard library copy started...\n");
    start = clock();
    library_copy(source, "library_copy.txt");
    end = clock();

    library_time = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Standard library copy completed.\n");
    printf("Time taken: %.6f seconds\n\n", library_time);

    printf("=== Performance Comparison ===\n");
    printf("Low-level I/O : %.6f seconds\n", low_time);
    printf("Library I/O   : %.6f seconds\n", library_time);

    return 0;
}

```

Source.txt

```

GNU nano 8.7.1
This is a test file for Practical 9.
This program demonstrates low-level Linux file I/O.
It uses open, read, write, lseek and close.
It also compares the performance with fopen, fread and fwrite.

```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week9
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ nano file_copy.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ nano source.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ gcc file_copy.c -o file_copy
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ ls
file_copy  file_copy.c  source.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ ./file_copy
=== File Copy Performance Test ===

Low-level I/O copy started...
Low-level copy completed.
Time taken: 0.001542 seconds

Standard library copy started...
Standard library copy completed.
Time taken: 0.000783 seconds

=== Performance Comparison ===
Low-level I/O : 0.001542 seconds
Library I/O   : 0.000783 seconds
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ ls -l source.txt low_level_copy.txt library_copy.txt
-rwxrwxrwx 1 ananya ananya 197 Sep 26 03:33 library_copy.txt
-rwxrwxrwx 1 ananya ananya 197 Sep 26 03:33 low_level_copy.txt
-rwxrwxrwx 1 ananya ananya 197 Sep 26 03:32 source.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ cat low_level_copy.txt
This is a test file for Practical 9.
This program demonstrates low-level Linux file I/O.
It uses open, read, write, lseek and close.
It also compares the performance with fopen, fread and fwrite.

ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ cat library_copy.txt
This is a test file for Practical 9.
This program demonstrates low-level Linux file I/O.
It uses open, read, write, lseek and close.
It also compares the performance with fopen, fread and fwrite.

ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ █
```

OBSERVATION:

The file copy utility was successfully implemented using low-level Linux I/O system calls such as `open()`, `read()`, `write()`, `lseek()`, and `close()`. The same file was also copied using standard C library functions including `fopen()`, `fread()`, `fwrite()`, and `fclose()`. Both copied files were created successfully and their contents matched the original source file. The execution times were measured for both approaches; in this test, the standard library method completed faster than the low-level I/O method. This demonstrates the difference between direct system-call-based I/O and buffered standard-library I/O.

Task - 2:

Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shell implements I/O redirection.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int input_fd;
    int output_fd;
    char buffer[100];
    ssize_t bytes_read;

    printf("=== I/O Redirection using dup2() ===\n");

    /* Open input file */
    input_fd = open("source.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("Error opening source.txt");
        return 1;
    }

    printf("Input file opened successfully.\n");

    /* Redirect standard input to source.txt */
    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input failed");
        close(input_fd);
        return 1;
    }

    close(input_fd);

    /* Open output file */
    output_fd = open("redirected_output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0)
    {
        perror("Error opening redirected_output.txt");
        return 1;
    }

    /* Redirect standard output to redirected_output.txt */
```

```

/* Redirect standard output to redirected_output.txt */
if (dup2(output_fd, STDOUT_FILENO) < 0)
{
    perror("dup2 output failed");
    close(output_fd);
    return 1;
}

close(output_fd);

/* Read from redirected standard input */
while ((bytes_read = read(STDIN_FILENO, buffer,
                          sizeof(buffer) - 1)) > 0)
{
    buffer[bytes_read] = '\0';

    /* Write to redirected standard output */
    write(STDOUT_FILENO, buffer, bytes_read);
}

return 0;
}

```

OUTPUT:

```

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week9
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ nano io_redirect.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ gcc io_redirect.c -o io_redirect
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ ls
file_copy file_copy.c io_redirect io_redirect.c library_copy.txt low_level_copy.txt source.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ ./io_redirect
=== I/O Redirection using dup2() ===
Input file opened successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ cat redirected_output.txt
This is a test file for Practical 9.
This program demonstrates low-level Linux file I/O.
It uses open, read, write, lseek and close.
It also compares the performance with fopen, fread and fwrite.

ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$ cmp source.txt redirected_output.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week9$

```

OBSERVATION:

The I/O redirection program was successfully implemented using file descriptors and the `dup2()` system call. The standard input was redirected from `source.txt`, while the standard output was redirected to `redirected_output.txt`. The program successfully transferred the contents from the input file to the output file. The generated output was verified using the `cmp` command, which produced no differences, confirming that the redirection was performed correctly. This demonstrates the basic mechanism used by the Linux shell to implement input (`<`) and output (`>`) redirection.